Training Dynamics of Overparameterized Neural Networks

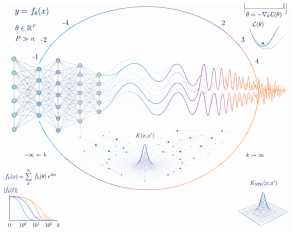
Shreyas Kalvankar

MSc Computer Science, TU Delft

Advisor: Dr. D. M. J. Tax, Pattern Recognition & Bioinformatics Group
Supervisor: Dr. A. Heinlein, Numerical Analysis Group

Committee Member: Prof. Dr. A. Papapantoleon, Applied Probability Group

TU Delft



Spectral Bias | NTK on S^1

Fourier Analysis

Finite-Scale Bounds

INTRODUCTION

Supervised Learning: Fitting data to a function

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

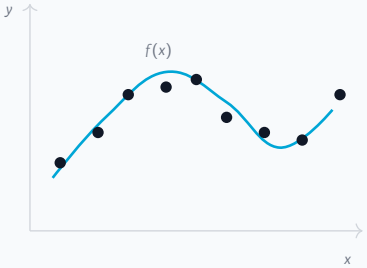
Model: a learned prediction function $f(x)$.

Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.



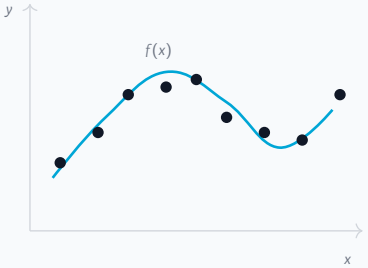
Supervised Learning: Fitting data to a function

Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n}\sum_{i=1}^{n}\big(\underbrace{f(x_i) - f^\star(x_i)}_{:= r_i}\big)^2$.



Supervised Learning: Fitting data to a function

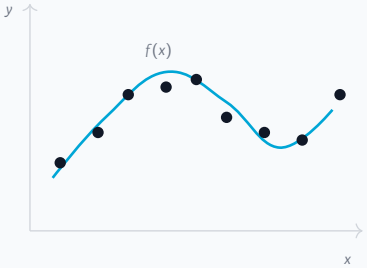
Data: n observations $(x_i, f^\star(x_i))$, examples of an unknown target function $f^\star$.

Model: a learned prediction function $f(x)$.

Aim: predict the target output $f^\star(x)$ for a new, unseen input x.

How: choose f so its predictions are close to the target function on the observed data, i.e. minimise $\mathcal{L}(f) = \frac{1}{n}\sum_{i=1}^{n}\big(\underbrace{f(x_i) - f^\star(x_i)}_{:= r_i}\big)^2$.

In practice: we cannot search over all possible functions, so we fix a parameterized family $f_\theta(x)$ and learn by adjusting the parameters θ. The parameters are the unknowns the data must pin down.



MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ:

$$f_\theta(x_i) \approx y_i, \text{ for } i = 1, \ldots, n.$$

A *solution*: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

MOTIVATION & RELEVANCE

Overparameterization: many ways to fit the same data

When many solutions fit the data, the learning process itself becomes important.

A linear analogy

3 equations, 3 unknowns $\Rightarrow$ one solution

3 equations, 4 unknowns $\Rightarrow$ infinitely many

independent equations, consistent system

One spare unknown, and the data no longer pins down the answer: a whole line of solutions fits.

The same situation in a neural network

Each training point is one constraint on the parameters θ:

$$f_\theta(x_i) \approx y_i, \text{ for } i = 1, \ldots, n.$$

A *solution*: a parameter setting with near-zero training error.

CIFAR-10 has **50k** training images (Krizhevsky, 2009);
WRN-28-10 has **36.5M** parameters and about **4%** test error (Zagoruyko & Komodakis, 2016).

Large networks can also fit random labels, so fitting the training set alone does not explain generalization (Zhang et al., 2017).

Empirical observation: models often learn simple structure before complex detail (Rahaman et al., 2019; Boursier & Flammarion, 2024).

MOTIVATION & RELEVANCE

Why study training behaviour?

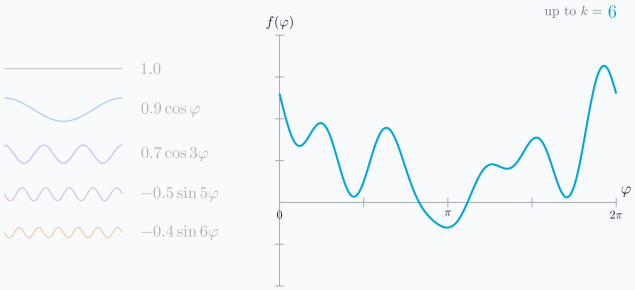
The broader goal is to understand learning, not just report a final test error.

- Once many fits are possible, improving a model is not only about making it larger; it is also about understanding what training is actually doing.
- This matters more as modern AI systems become expensive and energy-intensive to train: the compute used to train large-scale AI models is reported to double roughly every five months (Stanford AI Index, 2025); the Llama 3.1 family alone used 39.3M H100 GPU-hours for training (Meta AI, 2024).
- So before asking for a bigger model, we can ask: **what does gradient descent learn first?**

ILLUSTRATION

A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



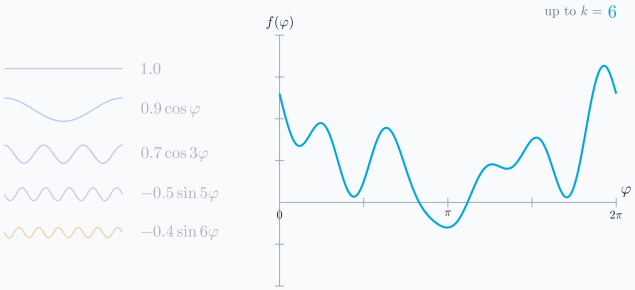
Frequency components

- Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- Low frequencies set the coarse shape; high frequencies add the fine detail.
- The target is a short sum of these pieces.

ILLUSTRATION

A concrete target

To watch that choice happen, take a target built from a coarse shape plus finer and finer detail.



Frequency components

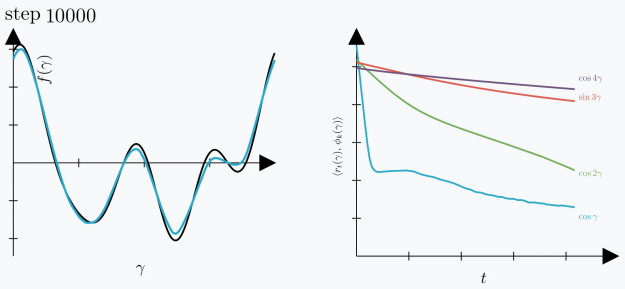
- Each piece (i.e. a *mode*) is one frequency, from slow waves to fast ones.
- Low frequencies set the coarse shape; high frequencies add the fine detail.
- The target is a short sum of these pieces.

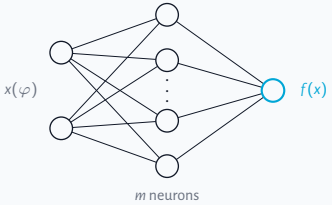
What gets learned first?

ILLUSTRATION

Watch one network learn it

Train a single-hidden-layer network on that target and watch the residual: what gets learned first?





$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right), \quad \sigma = \mathrm{ReLU}$$

The coarse shape appears first; the fine detail comes last.

Notice

- The fit climbs toward the target.
- Low-frequency modes die first.
- Fine detail fills in last.

THESIS

Why does gradient descent learn simple structure first?

This pattern has a name: the frequency principle, also called spectral bias (Xu et al., 2019).

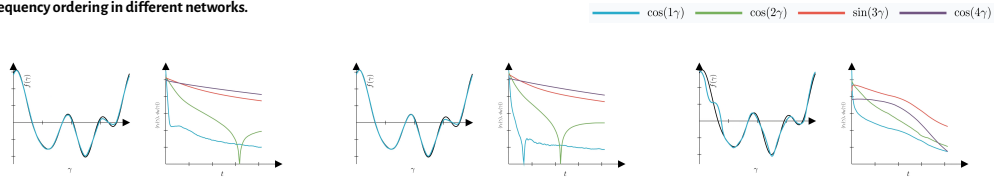
Thesis in one sentence

Derive the exact learning speed of every frequency in an idealized network, then quantify how far that exact picture persists in a realistic one: finitely many training points, finitely many neurons.

Three questions

1. What sets the learning speed of each frequency?
2. Does the answer survive when the network sees only n sampled points? *(RQ1)*
3. Does it survive with finitely many neurons, at the start of training and during it? *(RQ2)*

Frequency ordering in different networks.

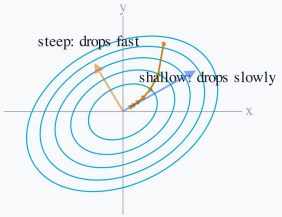


BACKGROUND

The error is a landscape

Seen from above, the contour rings are the bowl's height; training rolls toward the bottom.

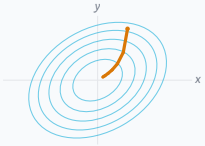
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i \left(f(x_i) - f^*(x_i)\right)^2$



Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$

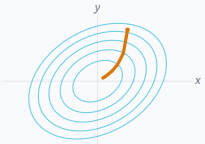


In x, y axes the two directions are coupled.

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

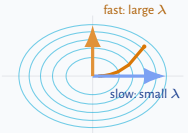
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



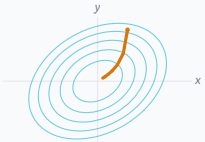
Along each axis the error falls on its own.

BACKGROUND

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

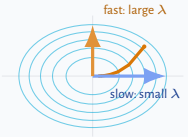
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

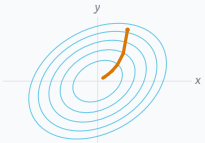
Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

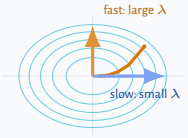
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

so it decays exponentially:

$$\alpha(t) = \alpha(0)\,e^{-\lambda t}$$

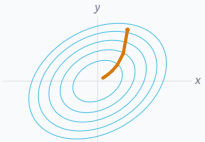
like cooling or half-life; λ is the rate.

BACKGROUND

Each direction has its own rate

Training makes the error small by walking downhill, along the steepest drop.

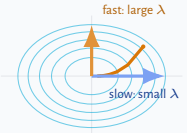
recall the loss: $\mathcal{L}(f) = \frac{1}{n} \sum_i (f(x_i) - f^*(x_i))^2$



In x, y axes the two directions are coupled.

$\Longrightarrow$

the bowl's own axes



Along each axis the error falls on its own.

Along one direction, the error falls in proportion to how much is left:

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

so it decays exponentially:

$$\alpha(t) = \alpha(0)\,e^{-\lambda t}$$

like cooling or half-life; λ is the rate.

Eigenvector: a direction the dynamics keeps separate, never mixing it with others. **Eigenvalue** λ: its rate, large is fast, small is slow.

What are these directions and rates for a neural network?

BACKGROUND

The rate becomes an operator: the Neural Tangent Kernel (Jacot et al., 2018)

The same law as before, with the single rate replaced by an operator.

recall the residual: $r(x) = f(x) - f^\star(x)$

One direction

$$\frac{d}{dt}\alpha(t) = -\lambda\,\alpha(t)$$

The error falls at rate λ, a single number.

A whole function

$$\partial_t r_t = -T_t\, r_t$$

The rate becomes an operator T_t, the network's tangent kernel (NTK).

If T_t were time-independent, diagonalizing it would split this back into one copy of the scalar law per direction k:

$$\dot{\alpha}_k = -\lambda_k\,\alpha_k$$

eigenfunctions are the directions · **eigenvalues** λ_k are the rates · together they form the **spectrum** of the time-independent T

So the question becomes: what is the spectrum of T?

SETUP

Making the dynamics analyzable

To read off a fixed set of directions and rates, T must be one fixed operator. Two things are in the way.

recall: $\partial_t r_t = -T_t\, r_t$

Time-dependent

T_t changes as the network trains, so its directions and rates drift.

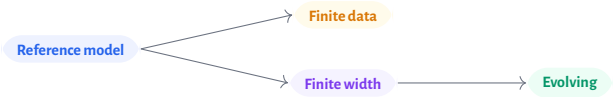
Solution: Take width to infinity to get a constant operator T (Jacot et al., 2018).

Finite scale

At finite data T depends on the sampled input points, so it changes on what it sees.

Solution: Consider using the continuum as input. (Jacot et al., 2018, Cao et al., 2019).

Strategy: build the clean reference, then relax each idealization separately.



The concrete model: regression on S^1

The unit circle gives a natural language for "coarse" and "fine": Fourier modes.

- Inputs are points on the unit circle with uniform distribution:

$$x(\varphi) = (\cos\varphi, \sin\varphi) \in S^1.$$

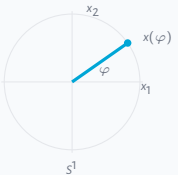
- Fourier basis functions are written compactly as

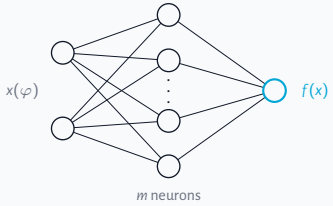
$$\phi_k \in \{1\} \cup \{\sin(k\varphi), \cos(k\varphi) : k \geq 1\}.$$

- The target is a Fourier sum with amplitudes c_k, the weights of each mode:

$$f^\star(\varphi) = \sum_k c_k \phi_k(\varphi), \qquad r_t = f_t - f^\star = \sum_k \alpha_k(t)\phi_k.$$

- The model is a one-hidden-layer ReLU network of width m, initialized with $w_i \sim \mathcal{N}(0, I_2)$, $a_i \sim \mathcal{N}(0,1)$, and $b_i = 0$.
- **Question:** what controls the decay of each $\alpha_k(t)$?





$$f_m(x;\theta) = \frac{1}{\sqrt{m}} \sum_{i=1}^{m} a_i \, \sigma\left(w_i^\top x + b_i\right), \quad \sigma = \mathrm{ReLU}$$

The ideal picture: a convolution operator on S^1

At infinite width and with continuum data, the NTK becomes fixed and rotation-invariant.

recall: fixed T gives $\dot{\alpha}_k = -\lambda_k \alpha_k$

With uniform measure $\mu(d\psi) = d\psi/(2\pi)$, the fixed operator acts by

$$(Tr)(\varphi) = \int_0^{2\pi} \Theta(\varphi - \psi)\, r(\psi)\, \frac{d\psi}{2\pi}.$$

Because this is a convolution on the circle, the Fourier modes are eigenfunctions:

$$T\phi_k = \lambda_k \phi_k, \qquad \alpha_k(t) = e^{-\lambda_k t}\alpha_k(0).$$

Explicit eigenvalues Thm. 5.2

$$\lambda_k = \begin{cases} \frac{1}{4} + \frac{3}{\pi^2}, & k = 0, \\ \frac{1}{4} + \frac{1}{\pi^2}, & k = 1, \\ \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

Interpretation

- Low frequencies $\Longrightarrow$ Small k: larger λ_k, faster decay.
- High frequencies $\Longrightarrow$ Large k: $\lambda_k \sim k^{-2}$, slower decay.

This exactly predicts the demo: coarse structure first, fine detail later.

Jacot et al., 2018; Cao et al., 2019

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Finite width

$$T \quad \rightsquigarrow \quad T_0^{(m)}$$

Replace the infinite-width operator by the finite-width tangent operator at initialization.

THESIS QUESTION

Does this spectral picture survive outside the ideal limit?

The ideal model is useful only if we understand what happens when its assumptions are relaxed.

Perturbation I

Finite samples

$$\mu \quad \rightsquigarrow \quad \frac{1}{n}\sum_{i=1}^{n}\delta_{\varphi_i}$$

Replace the continuous measure by n random points on S^1.

Question

Do Fourier modes still behave like eigenvectors of the sampled operator?

vs

Perturbation II

Finite width

$$T \quad \rightsquigarrow \quad T_0^{(m)}$$

Replace the infinite-width operator by the finite-width tangent operator at initialization.

Question

Does the finite-width tangent operator preserve the Fourier eigenspaces and learning rates?

Both perturbations break exact diagonalization; the goal is to quantify the finite-scale error (Bowman & Montúfar, 2022)

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \operatorname{span}\{1, \cos\varphi, \sin\varphi, \ldots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$\Lambda_K = \operatorname{diag}(\lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_K, \lambda_K)$, and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp\left(-Cn\varepsilon^2\right).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \operatorname{span}\{1, \cos\varphi, \sin\varphi, \ldots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$\Lambda_K = \operatorname{diag}(\lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_K, \lambda_K)$, and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp\left(-Cn\varepsilon^2\right).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width-m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$,

$$\mathbb{P}\left(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon\right) \leq 2d^2 \exp\left(-Cm\varepsilon^2\right).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K^{(m)} - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$$

Main results: the spectral picture survives finite scale

$$\mathcal{H}_K = \operatorname{span}\{1, \cos\varphi, \sin\varphi, \ldots, \cos(K\varphi), \sin(K\varphi)\}, \quad d = 2K + 1.$$

$\Lambda_K = \operatorname{diag}(\lambda_0, \lambda_1, \lambda_1, \ldots, \lambda_K, \lambda_K)$, and T_K denotes the relevant finite-scale operator on $\mathcal{H}_K$.

Finite samples

Thm. 5.5

Let T_K be the sampled operator on $\mathcal{H}_K$, using n uniform samples from $[0, 2\pi)$.

For every $\varepsilon > 0$,

$$\mathbb{P}(\|T_K - \Lambda_K\|_{\max} \geq \varepsilon) \leq 2d^2 \exp\left(-Cn\varepsilon^2\right).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{n}}\right).$$

Finite width

Thm. 5.6

Let $T_K^{(m)}$ be the width-m tangent operator on $\mathcal{H}_K$. Then $\mathbb{E}[T_K^{(m)}] = \Lambda_K$, and for every $\varepsilon > 0$,

$$\mathbb{P}\left(\|T_K^{(m)} - \Lambda_K\|_{\max} \geq \varepsilon\right) \leq 2d^2 \exp\left(-Cm\varepsilon^2\right).$$

Equivalently, with probability at least $1 - \delta$,

$$\|T_K^{(m)} - \Lambda_K\|_{\max} = O\left(\sqrt{\frac{\log(d/\delta)}{m}}\right).$$

What this gives us: a way to predict how the learning dynamics change with n and m. In particular, the finite-sample block tells us how to precondition the operator and reduce the frequency-wise gap in learning rates.

EVOLVING KERNEL

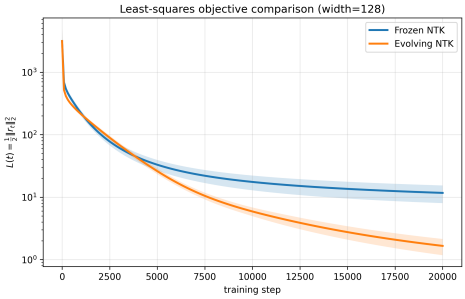
A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.

EVOLVING KERNEL

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.

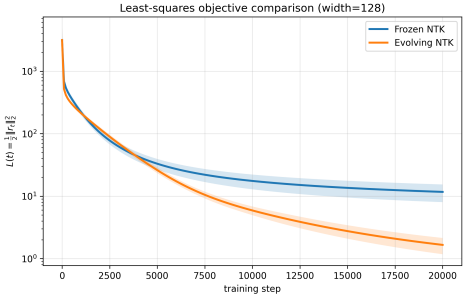


$m = 128$: evolving reaches lower training error

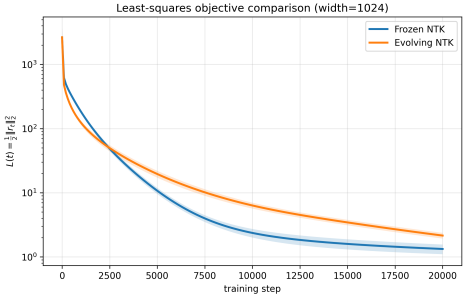
EVOLVING KERNEL

A finite-width effect not explained by the NTK at initialization

After controlling the operator at initialization, the next question is whether training changes the operator enough to matter.



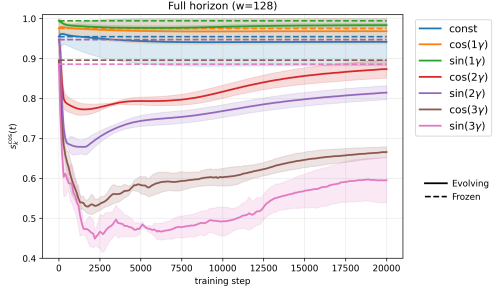
$m = 128$: evolving reaches lower training error



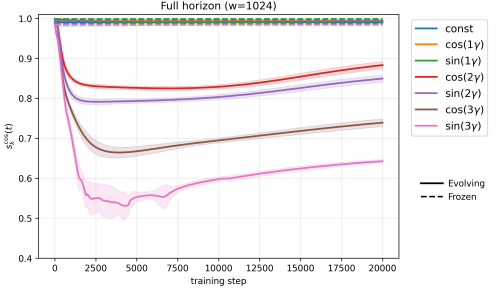
$m = 1024$: frozen reaches lower final training error

First check: does alignment improve?

If evolution helped by preserving Fourier geometry, the alignment curves should rise above their initial alignment. They do not.



$m = 128$: higher-frequency alignment drops sharply.

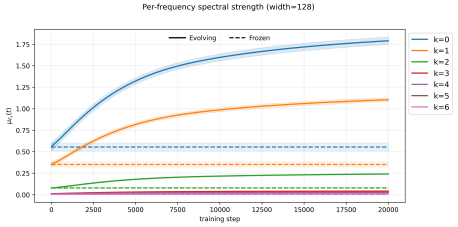


$m = 1024$: evolution still worsens high frequencies.

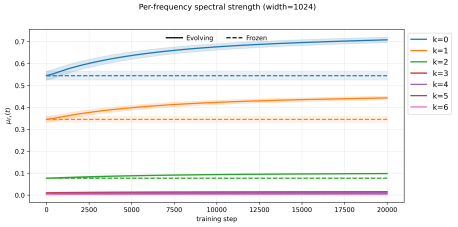
Alignment does not explain why the evolving kernel reaches lower training error at lower widths.

What changes: low-frequency strength rises

The moving kernel becomes stronger on some low-frequency blocks, even without better Fourier alignment.



$m = 128$: large increase in $k = 0$ and $k = 1$.



$m = 1024$: same direction of change, but much weaker.

The low-frequency preference is still visible, but the mechanism is not fully explained.

This is a diagnostic correlation, not a causal explanation. The effect is strongest at small width and much weaker at $m = 1024$.

CONCLUSION

What we can now say

The three questions from the start, answered.

1 What sets the learning speed of each frequency?

- Mode-wise rates are eigenvalues of the NTK; $\lambda_k \sim k^{-2}$, so low frequencies are learned first.

2 Does this survive with only n sampled points? *RQ1*

- Yes. Finite samples preserve the picture with explicit $O(n^{-1/2})$ control.

3 Does it survive with finitely many neurons, at init and during training? *RQ2*

- At initialisation, yes, with explicit $O(m^{-1/2})$ control. Once the kernel evolves the clean spectral picture no longer fully describes training: we observe increased low-frequency spectral strength alongside lower training error, but the cause is still open.

What we assumed, and what changes if we relax it

Assumption	What it gives us	If we relax it
Uniform measure on S^1	The NTK is a convolution, so the Fourier modes are exactly its eigenfunctions.	A non-uniform density or Gaussian inputs change the basis (e.g. Hermite polynomials), but eigenvalue size still sets the rates. The bounded-kernel proofs assume a compact domain, so they need rederiving for unbounded support.
ReLU activation	A closed-form spectrum with polynomial decay, $\lambda_k = O(k^{-2})$.	A smooth activation keeps the Fourier eigenbasis but decays faster, sharpening the bias. The fixed-kernel analysis carries over with the spectrum recomputed.
Frozen (lazy) kernel	Finite width is a controlled perturbation, concentrating at $O(m^{-1/2})$.	When the kernel evolves the initial spectrum no longer governs the dynamics; at small width we observe increased low-frequency strength alongside lower training error, with the cause still open. This regime is only studied empirically.
Fixed block $\mathcal{H}_K$	A controlled entrywise error on a chosen low-frequency block.	Over all modes, or with K growing in n, m, the entrywise bounds are not sharp where eigenvalues are small and close together; operator-norm tools would tighten them.

DISCUSSION & LIMITATIONS

Where this sits, and where it connects

- **An exact, concrete case study**
  - The NTK spectral-bias mechanism, usually argued asymptotically or shown empirically, made explicit on S^1, where every rate is a computable eigenvalue (Jacot et al., 2018; Rahaman et al., 2019).
- **Finite scale, not only the limit**
  - The infinite-width NTK is often treated as a purely asymptotic object; we add non-asymptotic bounds on a low-frequency block, alongside work probing finite-width corrections and the kernel picture's limits (Hanin & Nica, 2019; Vyas et al., 2022).
- **A baseline, not the whole story**
  - The fixed kernel predicts the rates while it stays lazy, but is not a full account once features are learned; this work marks where that reduction stops (Chizat et al., 2019).
- **Connects to practice**
  - The same spectrum PINNs rescale to fix high-frequency failure (Wang et al., 2022), and that Fourier features and SIREN reshape for coordinate networks (Tancik et al., 2020; Sitzmann et al., 2020).

Still, a result on the uniform circle is far from a guarantee about a deployed model.

Fourier structure survives finite scale, but kernel evolution changes the story.

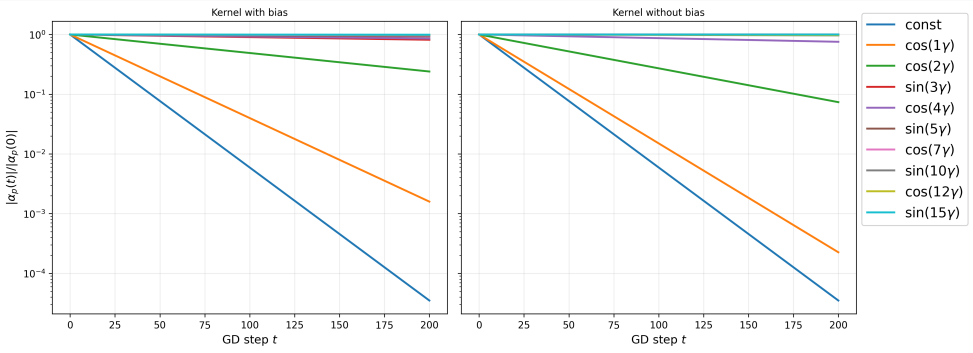
In the lazy regimes it predicts the rates; once the kernel evolves we observe increased low-frequency spectral strength alongside lower training error, but the cause is still open.

Questions?

[acknowledgements placeholder]

Backup

Continuum prediction: spectral bias



NTK: from parameter updates to an operator on functions

Gradient flow on the weights induces a linear flow on the residual, governed by an operator built from the network's own gradients.

Setup

Least-squares loss on $L^2(\mu)$, with residual $r_t = f_t - y$:

$$\mathcal{L}(\theta) = \tfrac{1}{2}\|f_m(\cdot;\theta) - y\|^2_{L^2(\mu)}.$$

The empirical NTK is the gradient inner product

$$\Theta_t^{(m)}(\varphi, \psi) = \big\langle \nabla_\theta f_t(\varphi), \nabla_\theta f_t(\psi) \big\rangle,$$

with operator $(T_t^{(m)} f)(\varphi) = \int \Theta_t^{(m)}(\varphi, \psi) f(\psi)\, d\mu(\psi)$.

Residual flow

Gradient flow on θ and the chain rule give a linear flow on the residual:

$$\partial_t r_t = -T_t^{(m)} r_t, \qquad \tfrac{d}{dt}\tfrac{1}{2}\|r_t\|^2 = -\langle r_t, T_t^{(m)} r_t \rangle.$$

Two regimes inside one equation:

- Frozen: replace $T_t^{(m)}$ by $T_0^{(m)}$ (kernel regression on the initial features).
- Evolving: keep the full time dependence of $T_t^{(m)}$.

As $m \to \infty$, $T_0^{(m)} \to T$, the deterministic continuum operator.

Finite-scale bounds: precise statements

$\mathcal{H}_K$: span of the Fourier modes up to frequency K, $d = 2K + 1$; Φ: those modes evaluated at the n samples; $\|\cdot\|_{\max}$: entrywise max. Main-slide notation: $T_K^{(n)}$ corresponds to H, and $T_K^{(m)}$ corresponds to $B_m^{(K)}$.

Finite samples

Thm. 5.5

Sampled kernel, Gram, and compressed operator:

$$A_{ij} = \tfrac{1}{n} T(\theta_i, \theta_j), \qquad G = \tfrac{1}{n} \Phi^\top \Phi, \qquad H = \tfrac{1}{n} \Phi^\top A \Phi$$

With probability at least $1 - \delta$:

$$\|G - I\|_{\max} \le C_1 \sqrt{\frac{\log(d/\delta)}{n}} \qquad \text{(orthogonality)}$$

$$\|H - \Lambda^{(n)}\|_{\max} \le C_2 \sqrt{\frac{\log(d/\delta)}{n}} \qquad \text{(eigenvalues)}$$

$$\|A\Phi - \Phi\Lambda^{(n)}\|_{\max} \le C_3 \sqrt{\frac{\log(nd/\delta)}{n}} \qquad \text{(kernel action)}$$

$\Lambda^{(n)}$: ideal eigenvalues with an $O(1/n)$ correction; $\mathbb{E}[H] = \Lambda^{(n)}$ (Prop. 5.4).

Frozen finite width

Thm. 5.6

Compress the width-m tangent operator at initialization to $\mathcal{H}_K$:

$$B_m^{(K)} = P_K T_0^{(m)} P_K, \qquad \Lambda_K = P_K T P_K$$

Unbiased at every width, and concentrating:

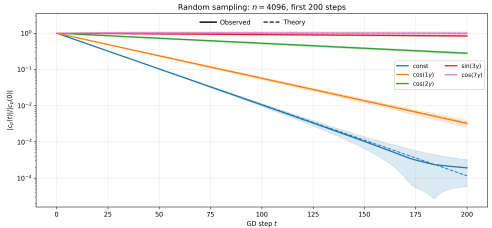
$$\mathbb{E}\, B_m^{(K)} = \Lambda_K$$

$$\|B_m^{(K)} - \Lambda_K\|_{\max} \le C_4 \sqrt{\frac{\log(d/\delta)}{m}}$$

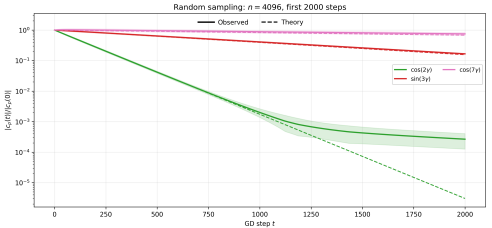
with probability at least $1 - \delta$; sub-exponential regime omitted.

Finite samples: observed mode decay tracks the eigenvalue prediction

$n = 4096$ uniform samples. Solid: observed normalised amplitude $|\alpha_p(t)|/|\alpha_p(0)|$. Dashed: diagonal finite-sample prediction from $\lambda_p^{(n)}$.



First 200 steps: low frequencies decay first, matching the prediction.

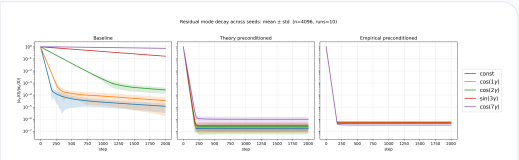


Longer horizon: the prediction holds until a component becomes small, then deviates.

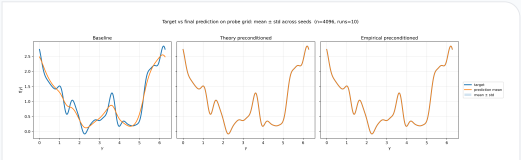
Sampling turns the exact continuum diagonal dynamics into an approximate low-frequency description, with deviations once a mode has nearly decayed.

Preconditioning: rescale each retained mode by its eigenvalue

The fixed-kernel dynamics are linear with per-mode rate λ_p. Rescaling the retained Fourier block flattens the rates. Theory: divide by predicted $\lambda_p^{(n)}$. Empirical: invert the observed block $C = G^{-1}H$.



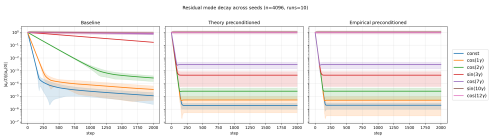
Both preconditioners flatten the retained-mode decay; empirical is most uniform.



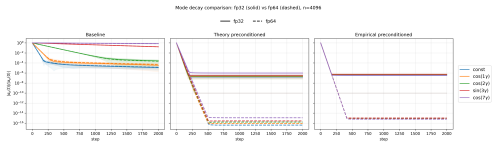
The retained components reach a much smaller error on the probe grid.

Preconditioning: the two floors that limit it

The preconditioner is built only on the retained block, and is solved in finite precision. Each limit shows up as a different plateau.



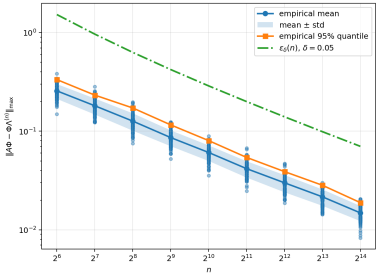
Out-of-block modes ($k = 10, 12$) stay near their start; in-block modes plateau higher from coupling.



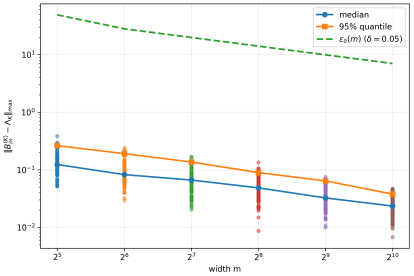
fp32 (solid) vs fp64 (dashed): the preconditioned plateau drops by orders of magnitude, so that floor is numerical ($\sim \kappa\varepsilon_{\mathrm{mach}}$).

The block correction is explicit and cheap, but acts only inside the retained block and bottoms out at the conditioning-set precision floor.

Concentration bounds: the constants, and where the magnitude bound is loose

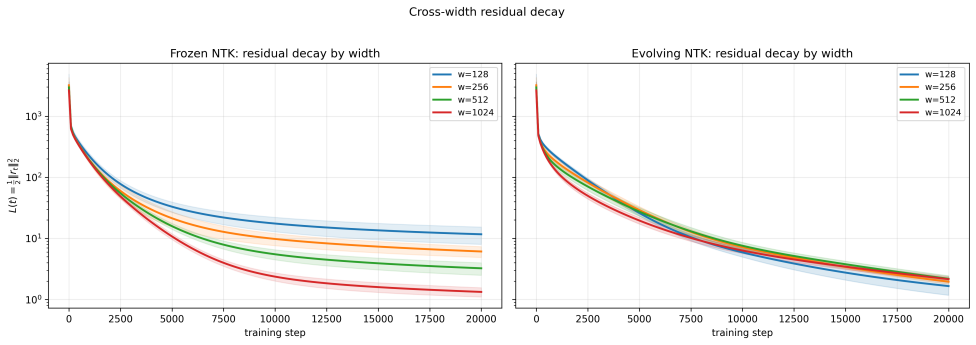


finite samples: action error decreases with n

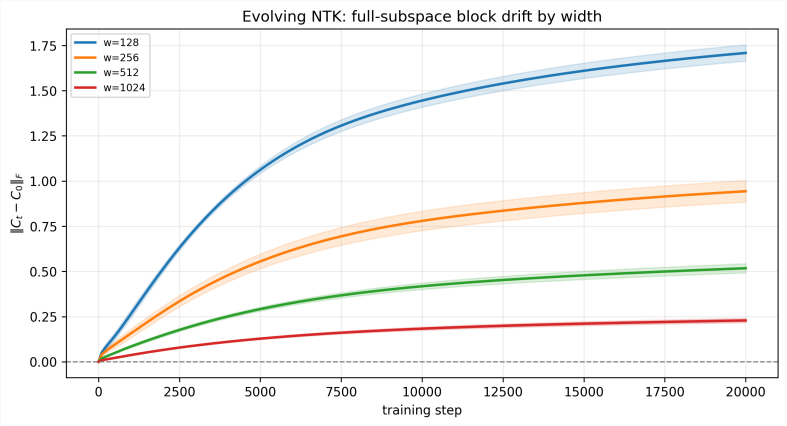


frozen finite width: block error decreases with m

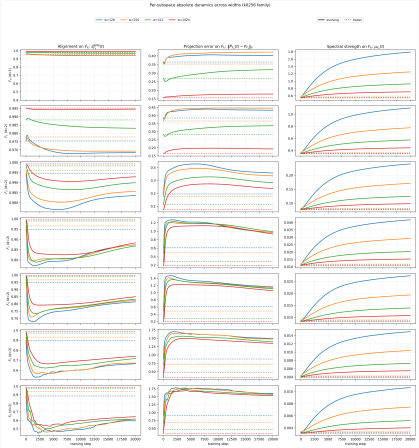
Frozen versus evolving: cross-width view



Kernel movement is a small-width effect



Full per-subspace evolving-kernel diagnostics



Computing the eigenvalues λ_k

The eigenvalues are the Fourier coefficients of the convolution kernel $\Theta(\delta)$.

From gradients to a closed-form kernel

At init ($a_i, w_i \sim \mathcal{N}(0, I)$, $b_i = 0$), the width-m NTK concentrates on

$$\Theta(x, y) = K_0 + (x^\top y + 1)K_1,$$

with the ReLU expectations (angle δ, $\cos\delta = x^\top y$)

$$K_1 = \tfrac{\pi - \delta}{2\pi}, \quad K_0 = \tfrac{1}{2\pi}\left(\sin\delta + (\pi - \delta)\cos\delta\right),$$

so that

$$\Theta(\delta) = \tfrac{1}{2\pi}\left(\sin\delta + 2(\pi - \delta)\cos\delta + (\pi - \delta)\right).$$

Convolution $\Rightarrow$ Fourier diagonal

Θ depends only on $\delta = \varphi - \psi$, so T is a convolution and

$$\lambda_k = \frac{1}{\pi}\int_0^\pi \Theta(\delta)\cos(k\delta)\, d\delta.$$

The parity integral $\int_0^\pi (\pi - \delta)\cos(m\delta)\, d\delta = \frac{1-(-1)^m}{m^2}$ splits even and odd k:

$$\lambda_0 = \tfrac{1}{4} + \tfrac{3}{\pi^2}, \quad \lambda_1 = \tfrac{1}{4} + \tfrac{1}{\pi^2},$$

$$\lambda_k = \begin{cases} \frac{k^2+3}{\pi^2(k^2-1)^2}, & k \geq 2 \text{ even}, \\ \frac{1}{\pi^2 k^2}, & k \geq 3 \text{ odd}. \end{cases}$$

The $k = 2$ alignment anomaly

λ_2 follows the generic even-k formula normally. The anomaly is in the geometry: at finite width, $k = 2$ is unusually well aligned with its Fourier subspace.

Observation

- Projector error grows with frequency, as expected, with one exception: $k = 2$ has the smallest error among the displayed frequencies.
- It is also the best aligned, sitting above its neighbours rather than on the decreasing trend.

A tentative explanation

- $k = 0$ gets a direct eigenvalue from the bias; $k = 1$ is tied to the coordinate embedding and is the only odd mode in the no-bias kernel.
- The finite-width perturbation may therefore act differently on $k = 0, 1$ than on higher blocks, leaving $k = 2$ relatively favoured.

This is an empirical observation, not a theoretical claim; the mechanism is not investigated further.

Eigenvalue ordering: where it holds, and what it depends on

Because the domain and measure are rotation-symmetric, the eigenvalue ordering is also a frequency ordering. The trainable bias is what keeps it strict.

The spectrum is strictly decreasing

$\lambda_0 \approx 0.554$ $\lambda_1 \approx 0.351$
$\lambda_2 \approx 0.079$ $\lambda_3 \approx 0.011$
$\lambda_4 \approx 0.0086$ $\lambda_5 \approx 0.0041$

Two interleaved subsequences (even and odd k), both $O(k^{-2})$, with $\lambda_k > \lambda_{k+1}$ throughout.

What the ordering depends on

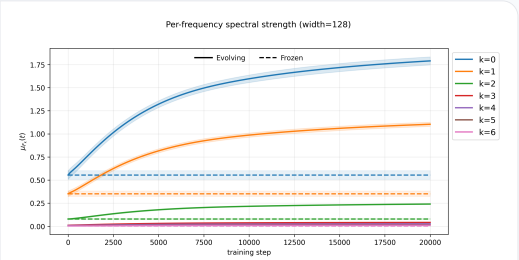
- The gap between $k = 0, 1$ and the rest comes from the constant and the coordinate embedding.
- The odd modes $k \geq 3$ are nonzero *only* because of the trainable hidden bias. Without it, $\lambda_k^{(\mathrm{nb})} = 0$ for odd $k \geq 3$, so those modes are null and never learned by the fixed kernel.
- Even $k \geq 2$ are unchanged by the bias.

Consistent with bias-free two-layer ReLU models not expressing odd harmonics beyond $k = 1$.

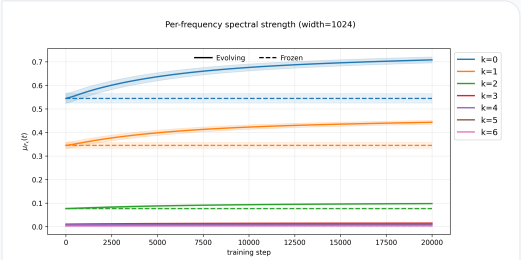
Why $k = 0, 1$ gain strength: the parameterisation hypothesis

With inputs $x(\varphi) = (\cos\varphi, \sin\varphi)$, the pre-activation $w_j^\top x + b_j = \|w_j\| \cos(\varphi - \psi_j) + b_j$ already contains $k = 0, 1$; higher harmonics appear only once the ReLU acts. The network may strengthen the directions it represents most directly.

Control: equal target amplitudes across all modes.



$m = 128$: strength still concentrates on $k = 0, 1$.

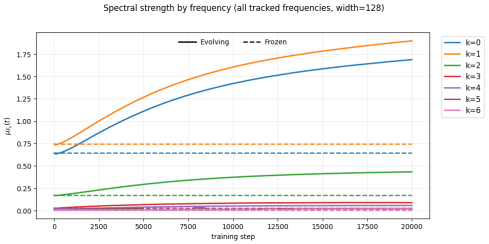


$m = 1024$: same direction, much weaker.

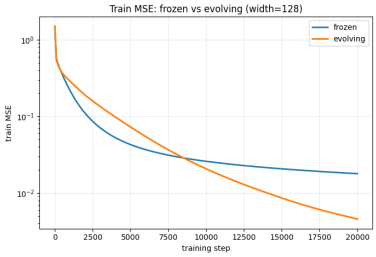
Even with equal target mass, the gain favours $k = 0, 1$, so it is not just tracking where the target places its mass.

A sharper probe: remove the lowest frequencies from the target

Target restricted to $k \geq 3$ (no $k = 0, 1, 2$ component). This separates two accounts: parameterisation (the gain stays on $k = 0, 1$, the directions the input embedding and bias represent directly) versus amplifying the largest residual (the gain follows the lowest *present* frequency).



$m = 128$: $k = 0, 1$ still gain the most strength, though absent from the target.

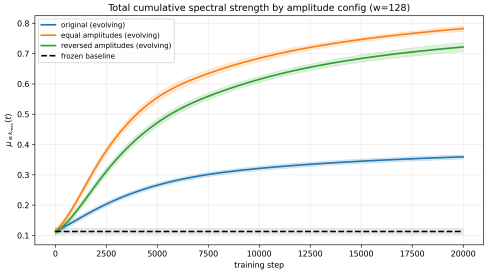


$m = 128$: evolving reaches lower training error than frozen, with $k = 0, 1, 2$ removed.

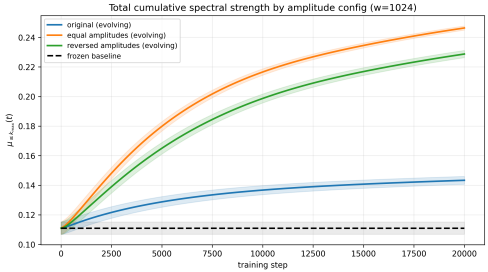
The strength gain still concentrates on $k = 0, 1$ although they are absent from the target, favouring the parameterisation account. This is one architecture on S^1 and a correlation, not an established cause.

Low-frequency strength gain is robust across amplitude configs

Cumulative low-frequency strength $\mu_{\leq k_{\max}}(t)$ for three amplitude profiles of the target: original, equal, and reversed (largest amplitudes on the highest frequencies).



$m = 128$: all three evolving configs rise well above the frozen baseline.



$m = 1024$: same ordering, smaller increase.

The increase is largest for the equal-amplitude target and still clear for the reversed one, which puts more mass on high frequencies. So the low-frequency strength gain is not a consequence of the original target placing larger amplitudes on lower frequencies.

Mean-field viewpoint: a complementary large-width limit

The NTK is one way to take width to infinity. The mean-field limit is another, and it is what makes the evolving-kernel regime natural.

NTK / lazy limit

- Track a *fixed* tangent operator at initialization.
- On S^1 it is diagonal in the Fourier basis, so learning is read off eigenvalues and eigenspaces.
- Features barely move; this is the regime our bounds control.

Mean-field limit

With scaling $f_\theta = \frac{1}{m}\sum_\alpha v_\alpha \varphi(w_\alpha^\top x)$, the output depends on the neurons only through their empirical measure

$$\mu_m = \frac{1}{m}\sum_\alpha \delta_{\theta_\alpha} \longrightarrow \mu_t.$$

- Training is a flow of the *distribution* of neurons, not a fixed kernel.
- This is the language for the feature-learning side where the kernel evolves.

References

- Rahaman, Baratin, Arpit, Draxler, Lin, Hamprecht, Bengio, Courville (2019). *On the Spectral Bias of Neural Networks*. ICML.
- Jacot, Gabriel, Hongler (2018). *Neural Tangent Kernel: Convergence and Generalization in Neural Networks*. NeurIPS.
- Bowman, Montúfar (2022). *Spectral Bias Outside the Training Set for Deep Networks in the Kernel Regime*. NeurIPS.
- Zhang, Bengio, Hardt, Recht, Vinyals (2017). *Understanding Deep Learning Requires Rethinking Generalization*. ICLR.